



Desarrollo Web Integral

Arquitectura del Sistema

Por:

Imanol Antonio de la Cruz

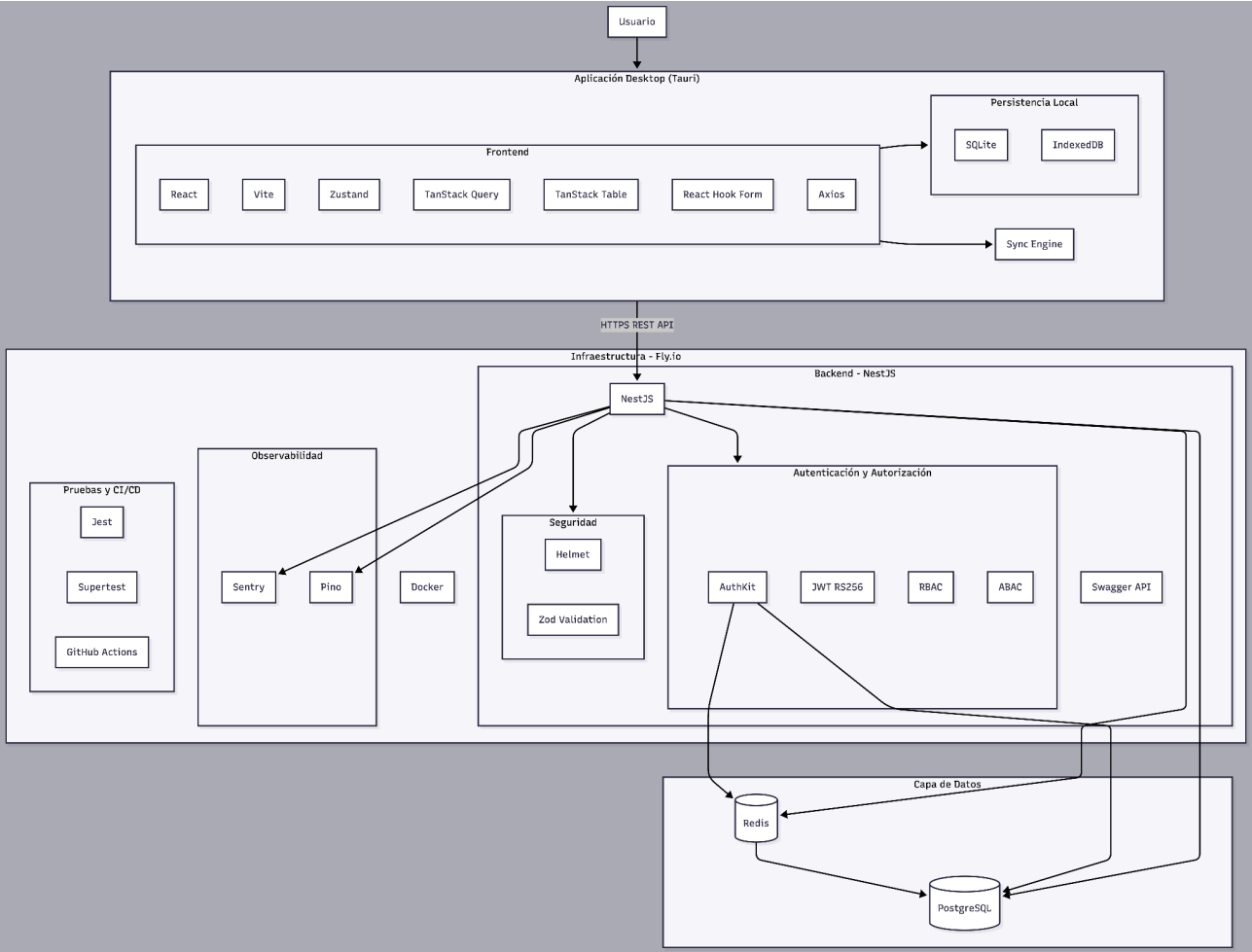
Claudia Espindola Lopez

Gracia Lopez Enrique Julián

Rodríguez Mendoza Fernando Spencer

Tehuacán, Puebla a 07 de Junio del 2026

Diagrama de Arquitectura



Modelo de datos

El modelo de datos centraliza todas las entidades del sistema POS SaaS. El aislamiento entre tenants se garantiza mediante Seguridad a nivel de fila (RLS) en PostgreSQL: cada sesión de base de datos aplica automáticamente el filtro por tenantId. Las operaciones offline quedan registradas en SyncEvent y se reconcilian con el servidor al restablecer la conexión a través del motor de sincronización.

Entidades del sistema

Gestión SaaS y multi-tenancy

Plan — Define los niveles de suscripción disponibles. Controla límites operativos y funcionalidades accesibles por tenant.

Campo	Tipo	Constraint	Descripción
id	String (cuid)	PK	Identificador único del plan
name	String		"Básico" "Pro" "Enterprise"
maxDevices	Int		Número máximo de dispositivos activos permitidos
maxUsers	Int		Número máximo de usuarios en el tenant
maxBranches	Int		Número máximo de sucursales habilitadas
priceMonthly	Decimal(10,2)		Precio mensual en MXN
features	Json		Flags de funcionalidades: { cfdi, advReports, api }
isActive	Boolean		Si false, el plan no está disponible para nuevas suscripciones
createdAt	DateTime		Fecha de creación del registro

Tenant — Representa una empresa cliente del SaaS. Es el contenedor raíz de todos sus datos operativos. Cada tenant está aislado mediante Row-Level Security (RLS) en PostgreSQL.

Campo	Tipo	Constraint	Descripción
id	String (cuid)	PK	Identificador único del tenant
name	String		Nombre legal o comercial de la empresa
slug	String	UQ	Identificador URL-friendly para subdominios futuros
planId	String	FK	Referencia a Plan contratado
status	TenantStatus		ACTIVE TRIAL SUSPENDED CANCELLED
trialEndsAt	DateTime		Fin del período de prueba (null si no aplica)

timezone	String		Zona horaria principal (e.g. "America/Mexico_City")
currency	String		Moneda base (e.g. "MXN")
logoUrl	String		URL del logotipo almacenado en R2/S3
createdAt	DateTime		Fecha de registro del tenant
updatedAt	DateTime		Última actualización (gestionado automáticamente por Prisma)

Subscription — Rastrea el estado del ciclo de pago del tenant. Se actualiza mediante webhooks de Stripe al procesar cobros, cancelaciones o renovaciones.

Campo	Tipo	Constraint	Descripción
id	String (cuid)	PK	Identificador único de la suscripción
tenantId	String	FK	Referencia a Tenant
planId	String	FK	Referencia a Plan contratado
stripeSubId	String	UQ	ID de suscripción en Stripe (null durante trial)
stripeCustomerId	String?		
status	String		active past_due canceled trialing
currentPeriodStart	DateTime		Inicio del período de facturación actual
currentPeriodEnd	DateTime		Fin del período; se renueva automáticamente
cancelAtPeriodEnd	Boolean		Si true, termina al fin del período sin renovarse
createdAt	DateTime		Fecha de creación de la suscripción

Usuarios y acceso

User — Empleado o administrador de un tenant. La autenticación se delega a AuthKit (WorkOS); no se almacena contraseña localmente.

Campo	Tipo	Constraint	Descripción
id	String (cuid)	PK	Identificador único del usuario
tenantId	String	FK	Referencia a Tenant
authkitUserId	String	UQ	ID del usuario en AuthKit — clave de unión con WorkOS
email	String		Correo electrónico del empleado
name	String		Nombre completo para mostrar en interfaz y reportes
role	Role		ADMIN MANAGER CASHIER — determina permisos RBAC
branchId	String	FK	Sucursal asignada (null = acceso a todas las sucursales)
isActive	Boolean		Desactivar sin eliminar preserva el historial de ventas
lastLoginAt	DateTime		Fecha del último inicio de sesión exitoso
createdAt	DateTime		Fecha de alta del usuario en el sistema
UNIQUE		UQ	(tenantId, email) — email único por tenant

Session — Refresh tokens activos por usuario. Permite revocar sesiones individuales por dispositivo sin afectar otras sesiones del mismo usuario.

Campo	Tipo	Constraint	Descripción
id	String (cuid)	PK	Identificador único de la sesión
userId	String	FK	Referencia a User
tenantId	String	FK	Tenant del usuario (optimiza consultas de revocación masiva)

refreshToken	String	UQ	Token opaco almacenado como hash SHA-256; nunca en claro
deviceId	String	FK	Dispositivo desde donde se inició la sesión
expiresAt	DateTime		Expiración del refresh token (default: 30 días)
userAgent	String		Información del cliente (navegador, versión de app)
createdAt	DateTime		Fecha de creación de la sesión

Device — Terminal física o virtual registrada en el sistema. El plan limita el número de dispositivos activos simultáneos. Incluye un token offline firmado con RS256 para operar sin conexión.

Campo	Tipo	Constraint	Descripción
id	String (cuid)	PK	Identificador único del dispositivo
tenantId	String	FK	Referencia a Tenant
branchId	String?	FK	Sucursal donde opera normalmente
name	String		"Caja 1", "Terminal entrada" — nombre legible para el operador
fingerprint	String		Hash del hardware o navegador para identificación única
isActive	Boolean		Solo los dispositivos activos cuentan en el límite del plan
offlineTokenVersion	Int	@default(0)	JWT RS256 firmado para operar hasta N días sin internet
offlineTokenExpiresAt	DateTime?		Fecha de expiración del token offline
lastSeenAt	DateTime?		Última actividad registrada del dispositivo
createdAt	DateTime		Fecha de registro del dispositivo
UNIQUE		UQ	(tenantId, fingerprint) — un hardware no se registra dos veces en el mismo tenant

Estructura comercial

Branch — Sucursal o punto de venta del tenant. El inventario y las ventas se pueden reportar de forma independiente por sucursal.

Campo	Tipo	Constraint	Descripción
id	String (cuid)	PK	Identificador único de la sucursal
tenantId	String	FK	Referencia a Tenant
name	String		"Sucursal Centro", "Local Norte"
address	String?		Dirección física completa incluyendo colonia y ciudad
phone	String?		Teléfono de contacto de la sucursal
timezone	String?		Zona horaria local (sobreescribe la del tenant si se define)
isActive	Boolean		Sucursales inactivas conservan su historial de ventas

Category — Agrupación de productos del catálogo. Facilita la navegación en el POS y la generación de reportes por segmento de negocio.

Campo	Tipo	Constraint	Descripción
id	String (cuid)	PK	Identificador único de la categoría
tenantId	String	FK	Referencia a Tenant
name	String		"Bebidas", "Electrónicos", "Ropa"
color	String?		Color hex para el grid del POS (e.g. "#3b82f6")

icon	String?		Nombre del ícono Tabler Icons para display en la interfaz
UNIQUE		UQ	(tenantId, name) — nombre de categoría único por tenant

Inventario

Product — Elemento del catálogo de productos del tenant. El precio base puede sobreescribirse por sucursal en ProductBranch. El costo permite calcular margen de ganancia en reportes.

Campo	Tipo	Constraint	Descripción
id	String (cuid)	PK	Identificador único del producto
tenantId	String	FK	Referencia a Tenant
sku	String		Código interno del producto (obligatorio, alfanumérico)
barcode	String?		Código de barras EAN-13 / UPC-A para escáner de pistola
name	String		Nombre del producto mostrado en el POS y reportes
description	String?		Descripción larga para catálogo digital o landing page
price	Decimal(10,2)		Precio de venta base en la moneda del tenant
cost	Decimal(10,2)?		Costo de compra; se usa para calcular margen en reportes
taxRate	Decimal(6,4)		Tasa de impuesto: 0.1600 = 16% IVA. 0.0000 = exento
categoryId	String?	FK	Referencia a Category
isActive	Boolean		Productos inactivos no aparecen en el POS pero se conservan en historial
imageUrl	String?		URL de la imagen almacenada en Cloudflare R2 o S3
createdAt	DateTime		Fecha de creación del producto

updatedAt	DateTime		Última modificación (gestionado automáticamente por Prisma)
UNIQUE		UQ	(tenantId, sku) — SKU único por tenant
INDEX			(tenantId, barcode) — búsqueda rápida por escáner de código de barras

ProductBranch — Stock e inventario específico por sucursal. Tabla de unión entre Product y Branch. Permite precios diferenciados y alertas de bajo stock independientes por ubicación.

Campo	Tipo	Constraint	Descripción
productId	String	PK / FK	Parte de la llave compuesta; referencia a Product
branchId	String	PK / FK	Parte de la llave compuesta; referencia a Branch
stock	Int		Unidades disponibles actualmente en esta sucursal
minStock	Int		Umbral para alerta de bajo stock (default: 5 unidades)
price	Decimal(10,2)?		Precio local — sobrescribe Product.price cuando está definido
PK			(productId, branchId) — un registro por combinación producto-sucursal

StockMovement — Log inmutable de todos los movimientos de inventario. Cada venta, compra, ajuste o devolución genera un registro. El stock actual es la suma de todos los movimientos.

Campo	Tipo	Constraint	Descripción
id	String (cuid)	PK	Identificador único del movimiento
tenantId	String	FK	Referencia a Tenant
productId	String	FK	Producto afectado por el movimiento
branchId	String	FK	Sucursal donde ocurrió el movimiento

type	StockMovementType		SALE PURCHASE ADJUSTMENT RETURN TRANSFER
quantity	Decimal(10,3)		Positivo = entrada de stock. Negativo = salida del almacén
reason	String?		Descripción opcional del motivo (merma, robo, error, etc.)
userId	String	FK	Usuario que realizó o autorizó el movimiento
saleId	String?	FK	Referencia a Sale (solo si type = SALE o RETURN)
createdAt	DateTime		Fecha e hora del movimiento — campo inmutable, no editable

Transacciones

Sale — Registro de cada transacción comercial. El campo `offlineId` garantiza la idempotencia durante la sincronización. Los totales se calculan y persisten en el momento de la venta.

Campo	Tipo	Constraint	Descripción
id	String (cuid)	PK	Identificador único generado por el servidor al sincronizar
tenantId	String	FK	Referencia a Tenant
branchId	String	FK	Sucursal donde se realizó la transacción
deviceId	String?	FK	Terminal desde donde se procesó la venta
userId	String	FK	Cajero o usuario que registró la venta
status	SaleStatus		COMPLETED CANCELLED REFUNDED PARTIAL
subtotal	Decimal(10,2)		Suma de (unitPrice × quantity) de todos los ítems
taxAmount	Decimal(10,2)		Impuesto total calculado en el momento de la venta
discountAmount	Decimal(10,2)		Descuento total aplicado sobre la venta completa

total	Decimal(10,2)		subtotal + taxAmount - discountAmount
notes	String?		Nota interna o referencia del cliente
cfidId	String?		Folio fiscal SAT — solo si se emitió comprobante CFDI
offlineId	String	UQ	UUID v4 generado en el cliente offline — garantiza idempotencia en sync
syncedAt	DateTime?		Fecha en que el servidor recibió y aplicó la venta
createdAt	DateTime	@default(now())	Fecha de la transacción — generada en el cliente, no en el servidor
INDEX			(tenantId, createdAt) — optimiza reportes por período de tiempo
INDEX			(tenantId, branchId) — optimiza filtrado por sucursal
occurredAt	DateTime		timestamp del cliente — cuándo ocurrió la venta

SaleItem — Cada línea de producto dentro de una venta. El precio unitario y la tasa de impuesto se copian como snapshot en el momento exacto de la venta para preservar la integridad histórica.

Campo	Tipo	Constraint	Descripción
id	String (cuid)	PK	Identificador único del ítem
saleId	String	FK	Referencia a la venta que contiene este ítem
productId	String	FK	Referencia al producto vendido
quantity	Int		Unidades vendidas de este producto en la transacción
unitPrice	Decimal(10,2)		Precio unitario al momento de la venta (snapshot histórico)
taxRate	Decimal(5,4)		Tasa de impuesto al momento de la venta (snapshot histórico)
discount	Decimal(10,2)		Descuento aplicado a esta línea específica

subtotal	Decimal(10,2)		(unitPrice × quantity) - discount — calculado y persistido
----------	---------------	--	--

Payment — Método de pago dentro de una venta. Una misma venta puede tener múltiples registros de pago para soportar pagos mixtos (efectivo + tarjeta, por ejemplo).

Campo	Tipo	Constraint	Descripción
id	String (cuid)	PK	Identificador único del pago
saleId	String	FK	Venta a la que pertenece este pago
method	PaymentMethod		CASH CARD TRANSFER QR CREDIT
amount	Decimal(10,2)		Monto cobrado con este método de pago
reference	String?		Número de autorización, referencia de transferencia, etc.
status	String		COMPLETED PENDING FAILED
createdAt	DateTime		Fecha y hora del pago

Sincronización offline

SyncEvent — Log de todos los eventos generados por clientes offline. Permite auditoría completa, replayabilidad y detección de conflictos durante la sincronización.

Campo	Tipo	Constraint	Descripción
id	String (cuid)	PK	Identificador único del evento de sincronización
tenantId	String	FK	Referencia a Tenant — permite filtrado masivo por empresa
deviceId	String	FK	Dispositivo que originó el evento offline
entityType	String		"sale" "product" "stock_movement" "payment"
entityId	String		ID del registro afectado — puede ser el offlineId del cliente

operation	SyncOp		CREATE UPDATE DELETE
payload	Json		Datos completos del registro en el momento del evento offline
clientTs	DateTime		Timestamp del cliente — base para resolución last-write-wins
serverTs	DateTime		Timestamp del servidor al recibir y procesar el evento
status	SyncStatus		PENDING APPLIED CONFLICT REJECTED DUPLICATE
conflictNote	String		Descripción técnica del conflicto detectado durante el merge
resolvedBy	String		"server" "client" "manual" — regla de resolución aplicada
INDEX			(tenantId, deviceId, status) — consulta de eventos pendientes por dispositivo

Relaciones entre entidades

Origen	Cardinalidad	Destino	Descripción
Plan	1 → N	Tenant	Un plan puede tener múltiples tenants suscritos
Plan	1 → N	Subscription	Historial de planes contratados
Tenant	1 → 1	Subscription	Un tenant tiene una suscripción activa a la vez
Tenant	1 → N	User	Un tenant tiene múltiples usuarios (empleados)
Tenant	1 → N	Branch	Un tenant puede tener múltiples sucursales
Tenant	1 → N	Device	Un tenant tiene múltiples terminales registradas
User	1 → N	Session	Un usuario puede tener múltiples sesiones activas

User	$N \rightarrow 1$	Branch	Cada usuario puede estar asignado a una sucursal específica
User	$1 \rightarrow N$	Sale	Un usuario realiza múltiples ventas a lo largo del tiempo
Device	$1 \rightarrow N$	Sale	Una terminal registra múltiples ventas en su operación
Device	$1 \rightarrow N$	Session	Un dispositivo puede tener sesiones de usuario asociadas
Branch	$1 \rightarrow N$	Sale	Una sucursal acumula múltiples ventas en su historial
Branch	$N \rightarrow M$	Product	Via ProductBranch — stock e inventario independiente por sucursal
Category	$1 \rightarrow N$	Product	Una categoría agrupa múltiples productos del catálogo
Product	$1 \rightarrow N$	SaleItem	Un producto puede aparecer en múltiples líneas de ventas distintas
Product	$1 \rightarrow N$	StockMovement	Cada movimiento de inventario referencia un producto específico
Sale	$1 \rightarrow N$	SaleItem	Una venta contiene una o más líneas de producto
Sale	$1 \rightarrow N$	Payment	Una venta puede pagarse con múltiples métodos (pago mixto)
Sale	$1 \rightarrow N$	StockMovement	Una venta genera movimientos de salida de stock automáticamente
Device	$1 \rightarrow N$	SyncEvent	Cada dispositivo genera eventos de sincronización al reconectar
Tenant	$1 \rightarrow N$	SyncEvent	Todos los eventos se filtran y aíslan por tenant

Enumeraciones

Enum	Valores	Descripción
------	---------	-------------

TenantStatus	ACTIVE · TRIAL · SUSPENDED · CANCELLED	Ciclo de vida del tenant. SUSPENDED bloquea el acceso sin eliminar datos.
Role	ADMIN · MANAGER · CASHIER	Roles del sistema RBAC. Determinan permisos de acceso a módulos y operaciones críticas.
SaleStatus	COMPLETED · CANCELLED · REFUNDED · PARTIAL	Estado de la transacción. PARTIAL indica pago incompleto o devolución parcial.
PaymentMethod	CASH · CARD · TRANSFER · QR · CREDIT	Método de pago aceptado en el punto de venta. Una venta puede combinar múltiples métodos.
StockMovementType	SALE · PURCHASE · ADJUSTMENT · RETURN · TRANSFER	Origen del movimiento de inventario para trazabilidad y auditoría contable.
SyncOp	CREATE · UPDATE · DELETE	Operación realizada offline que se replica al servidor durante la sincronización.
SyncStatus	PENDING · APPLIED · CONFLICT · REJECTED · DUPLICATE	Estado del procesamiento del evento. DUPLICATE indica que el offlineId ya existía en el servidor.

Decisiones de diseño

Row-Level Security (RLS) para aislamiento multi-tenant En lugar de schemas separados por empresa, se usa una única base de datos PostgreSQL con RLS activado. Al iniciar cada sesión se ejecuta `SET app.current_tenant = 'uuid'`, y las políticas RLS filtran automáticamente todos los registros. Esto simplifica las migraciones (una sola migración aplica a todos los tenants) y escala a miles de empresas sin overhead operativo adicional.

offlineId (UUID v4) para idempotencia en sincronización Cada venta generada sin conexión incluye un UUID v4 creado en el cliente antes de enviarse al servidor. El Sync Engine verifica si ya existe un registro con ese `offlineId` antes de aplicar el evento, haciendo la operación completamente idempotente. Esto permite reintentar la sincronización con exponential backoff sin riesgo de duplicar ventas en la base de datos.

Snapshot de precios en SaleItem — preservación del valor histórico El precio unitario y la tasa de impuesto se copian directamente en SaleItem en el momento exacto de la venta. Si el producto cambia de precio posteriormente, el historial de ventas permanece correcto e inmutable. Este es el patrón estándar en todos los sistemas POS de nivel empresarial para garantizar la integridad contable.

AuthKit (WorkOS) para autenticación — sin contraseñas locales La autenticación se delega completamente a AuthKit (WorkOS). Esto proporciona SSO, MFA y gestión segura de credenciales sin almacenar contraseñas en la base de datos del POS. El campo `authkitUserId` es la clave de unión entre el perfil de WorkOS y el registro `User` del sistema interno.

StockMovement como log inmutable — event sourcing de inventario El inventario no se modifica con `UPDATE` directo; cada operación genera un `StockMovement` en modo `append-only`. El stock actual se calcula como `SUM(quantity)` de todos los movimientos del producto en la sucursal. Este enfoque permite auditoría completa y reconstrucción del estado del inventario en cualquier fecha histórica.

Justificación del stack

Frontend

Se seleccionó Tauri para desarrollar la aplicación de escritorio multiplataforma con bajo consumo de recursos. La interfaz se construirá con React y Vite, permitiendo un desarrollo rápido y escalable. Para la gestión de estado y consumo de datos se utilizan Zustand, TanStack Query, React Hook Form y Axios, mejorando el rendimiento y la experiencia del usuario y desarrollador.

Backend

Se eligió NestJS por su arquitectura modular y escalable. La persistencia de datos se gestiona mediante Prisma ORM, mientras que Swagger proporciona documentación automática de la API.

Base de Datos

PostgreSQL funciona como base de datos backup en la nube. Redis se utiliza para caché y gestión de sesiones, mientras que SQLite e IndexedDB permiten el funcionamiento offline de la aplicación.

Autenticación y Autorización

La autenticación se implementa con AuthKit y JWT RS256, utilizando Access Tokens, Refresh Tokens y Token Rotation para mejorar la seguridad. La autorización se maneja con RBAC, permitiendo un control de acceso flexible basado en roles.

Infraestructura

La aplicación se despliega en Fly.io utilizando Docker, garantizando portabilidad, facilidad de despliegue y consistencia entre entornos.

APIs

Se implementa una API REST para la comunicación entre frontend y backend mediante solicitudes HTTP y respuestas en formato JSON. La documentación se genera automáticamente con Swagger.

Seguridad

La seguridad se fortalece mediante Helmet, validación de datos con Zod, configuración de CORS y control de solicitudes mediante Rate Limiting (gracias a NestJS), reduciendo riesgos de ataques y datos inválidos.

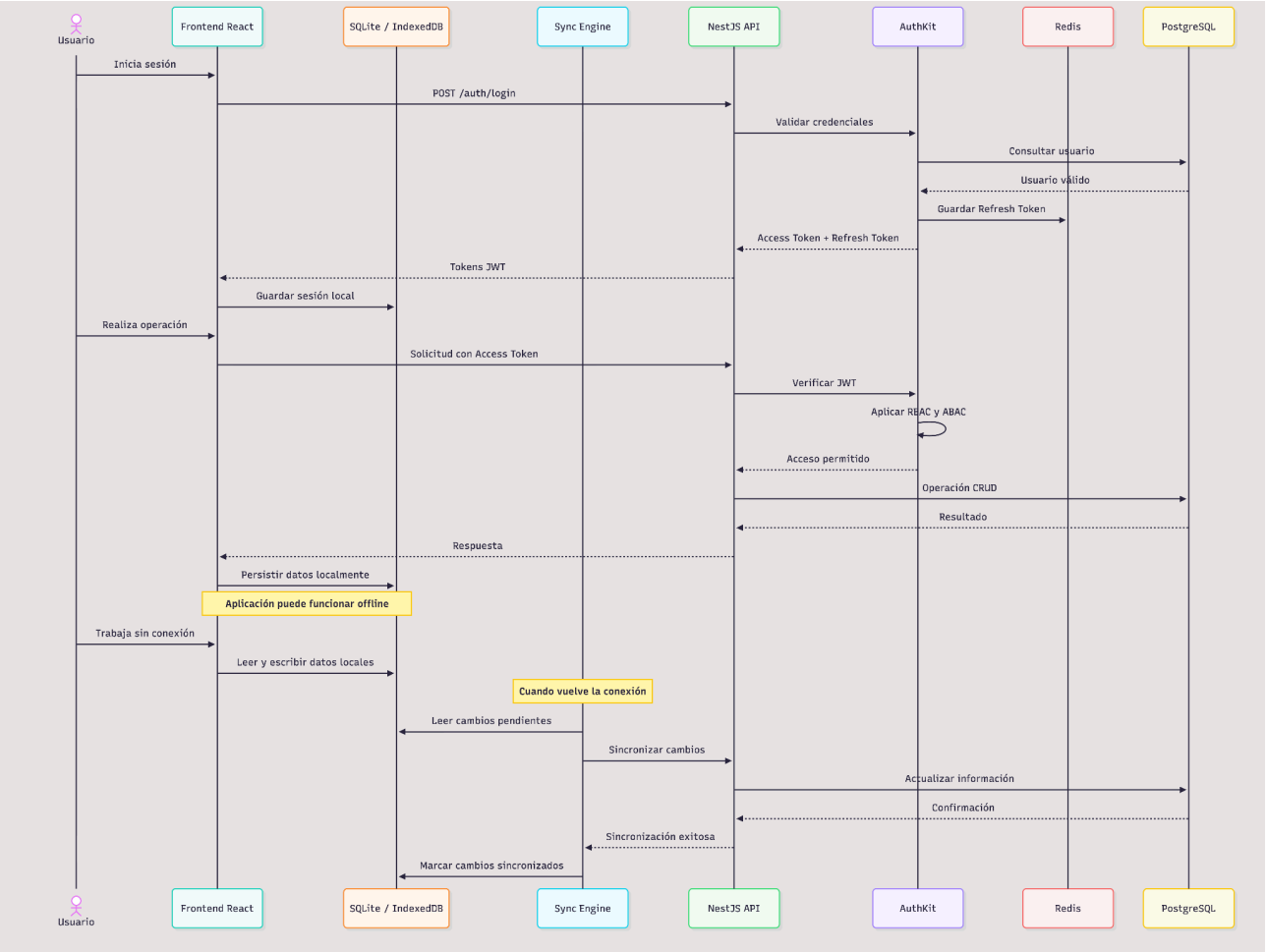
Pruebas

Se utilizan Jest para pruebas unitarias y Supertest para pruebas de integración y APIs. Además, GitHub Actions automatiza la ejecución de pruebas y validaciones dentro del proceso de integración continua.

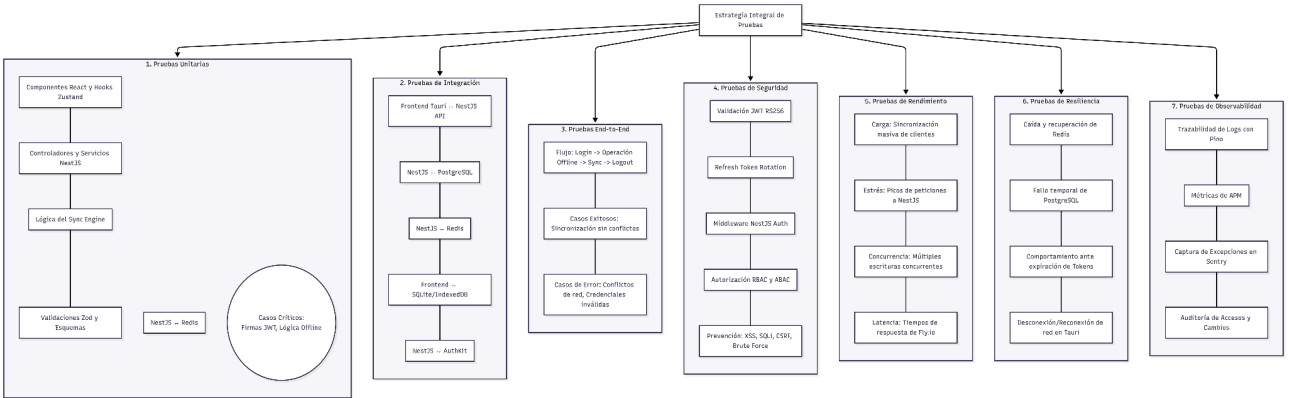
Observabilidad

Sentry permite monitorear errores en tiempo real y Pino registra eventos y operaciones del sistema, facilitando la detección y análisis de incidentes.

Flujo del sistema



Esquema de pruebas



1. Pruebas Unitarias

Las pruebas unitarias tienen como objetivo verificar el funcionamiento individual de cada componente del sistema. En EQUIPO CSS se aplicarían a los componentes React, hooks

de Zustand, controladores y servicios de NestJS, lógica del motor de sincronización y esquemas de validación definidos con Zod.

Herramientas: Jest y React Testing Library.

Resultado esperado: Cada módulo funciona correctamente de manera independiente y produce los resultados esperados ante entradas válidas e inválidas

2. Pruebas de Integración

Las pruebas de integración permiten validar la comunicación entre los distintos componentes del sistema. Se verificaría la interacción entre el frontend y la API, la comunicación de NestJS con PostgreSQL, Redis y AuthKit, así como el almacenamiento local mediante SQLite e IndexedDB.

Herramientas: Jest y Supertest.

Resultado esperado: Los módulos intercambian información correctamente y mantienen la consistencia de los datos.

3. Pruebas End-to-End (E2E)

Las pruebas End-to-End simulan el comportamiento real de un usuario dentro de la aplicación. Se evaluaría el flujo completo desde el inicio de sesión, la realización de operaciones, el trabajo en modo offline, la sincronización de datos y el cierre de sesión.

Herramientas: Playwright o Cypress.

Resultado esperado: El usuario puede completar todas las tareas previstas sin errores.

4. Pruebas de Seguridad

Estas pruebas buscan garantizar la protección de la información y el control de acceso al sistema. Se verificaría la validez de los JWT RS256, la rotación de Refresh Tokens, el funcionamiento de los middleware de autenticación y los mecanismos de autorización RBAC y ABAC.

Herramientas: JWT, AuthKit, Helmet y herramientas de análisis de seguridad.

Resultado esperado: Solo los usuarios autorizados pueden acceder a los recursos permitidos y el sistema resiste ataques comunes como XSS, SQL Injection, CSRF y fuerza bruta.

5. Pruebas de Rendimiento

Las pruebas de rendimiento permiten medir la capacidad de respuesta del sistema bajo diferentes niveles de carga. Se analizarían procesos como la sincronización masiva de clientes, picos de solicitudes hacia NestJS y operaciones concurrentes.

Herramientas: k6 o Artillery.

Resultado esperado: El sistema mantiene tiempos de respuesta aceptables y estabilidad durante cargas elevadas.

6. Pruebas de Resiliencia

Las pruebas de resiliencia evalúan la capacidad del sistema para recuperarse ante fallos. Se contemplan escenarios como la caída de Redis, fallos temporales de PostgreSQL, expiración de tokens y pérdida de conexión de red.

Resultado esperado: La aplicación continúa operando mediante sus mecanismos de recuperación y sincronización, minimizando el impacto para el usuario.

7. Pruebas de Observabilidad

Estas pruebas verifican la capacidad de monitorear y diagnosticar el comportamiento de la aplicación. Se revisaría la generación de logs mediante Pino, la captura de excepciones con Sentry, las métricas de rendimiento y la auditoría de accesos y cambios.

Herramientas: Pino y Sentry.

Resultado esperado: Los eventos importantes, errores y métricas quedan registrados para facilitar el monitoreo y mantenimiento del sistema.